Homework 3
Final Project First Report

Juan Manuel Pascual Navarro
March 4, 2026

Abstract

This first phase implements multiclass logistic regression and LDA from scratch using JAX, following ESL. Models are evaluated on GTZAN dataset (10 genres) with precision/recall/F1 metrics.

Index Terms

Music genre classification, GTZAN, logistic regression, linear discriminant analysis, JAX, ESL, maximum likelihood, Newton-Raphson, FMA.

INTRODUCTION

The goal of this final project is to develop a complete music genre classification system using the FMA (Free Music Archive) dataset through a machine learning pipeline. However, for this first submission, and in order to validate the correct implementation of the models, the GTZAN dataset has been used. GTZAN is smaller (1000 songs, 60 features, and 10 genres), unlike FMA, which, with 512 features, will be used to employ feature selection (a topic for later study). This allows for rapid iteration and debugging of the code for the first two models to be implemented in this submission.

This first report focuses on the implementation of linear and logistic classifier models, but unlike what was covered in class, it extends to multiclass classification following the guidelines of the book *The Elements of Statistical Learning* (ESL) by Hastie et al. Specifically, it implements the following models:

- Multi-class Logistic Regression (Chapter 4.4 of ESL)
- Linear Discriminant Analysis (LDA) (Chapter 4.3 of ESL)

The accuracy and performance of both classifiers are measured using the metrics of accuracy evaluation, recall, and the average F1 score across all classes.

METHODOLOGY

LDA was implemented according to the ESL formulation (Chapter 4.3), which assumes normal distributions with a common covariance matrix. The discriminant function for the class

$$\delta_k(x) = x^T\Sigma^{-1}\mu_k - \frac{1}{2}\mu_k^T\Sigma^{-1}\mu_k + log\pi_k$$

where $\mu_k$is the mean of the class k, Σ is the shared covariance matrix and $\pi_k$the priori probability. The pseudo-inverse was used to invert the covariance matrix, although theoretically it is assumed to be invertible.

```
import jax.numpy as jnp

class LDA:
    """
    Attempt of Linear Discriminant Analysis implementation as ESL (Chapter 4.3).
    Assuming that all classes share the same covariance matrix.
    """

    def __init__(self):
        self.classes = None # Unique class labels
        self.means = None # Class mean vectors _k
        self.priors = None # Class-prior probabilities (_k^ = N_k / N)
        self.cov = None # Shared covariance matrix
        self.cov_inv = None # Inverse of the covariance matrix

    def fit(self, X, y):
        # Unique class labels
        self.classes = jnp.unique(y)
        n_samples, n_features = X.shape
        K = len(self.classes)

```

```
        # Estimate class priors _k
        self.priors = jnp.array([
            jnp.sum(y == c) / n_samples
            for c in self.classes
        ])
        # Estimate class means _k
        self.means = jnp.array([
            jnp.mean(X[y == c], axis=0)
            for c in self.classes
        ])
        # Initializing shared covariance matrix
        cov = jnp.zeros((n_features, n_features))
        for i, c in enumerate(self.classes):
            Xc = X[y == c]
            centered = Xc - self.means[i]
            cov += centered.T @ centered
        cov /= (n_samples - K)
        self.cov = cov

        # Inverse covariance matrix ^{-1}
        self.cov_inv = jnp.linalg.inv(cov)

        return self

    def delta(self, X, k):
        """
        Discriminant function _k(x) for class k.
        """
        mu_k = self.means[k]
        term1 = X @ self.cov_inv @ mu_k
        term2 = - 0.5 * (mu_k @ self.cov_inv @ mu_k)
        term3 = jnp.log(self.priors[k])

        return term1 + term2 + term3

    def decision_rule(self, X):
        """
        Calculate _k(x) for each class.
        """
        rules = []
        for k in range(len(self.classes)):
            rules.append(self.delta(X, k))

        return jnp.stack(rules, axis=1)

    def predict(self, X):
        """
        Classify using
            argmax_k _k(x)
        """
        scores = self.decision_rule(X)
        idx = jnp.argmax(scores, axis=1)

        return self.classes[idx]
```

Following the ESL formulation (Chapter 4.4), the posterior probabilities are modeled using K-1 linear functions, taking the last class (K) as the reference.

$$log\frac{Pr(G=k \mid X=x)}{Pr(G=K \mid X=x)} = \beta_{k0} + \beta_k^T x, k = 1, \ldots, K-1$$

and probabilities:

$$Pr(G=k \mid X=x) = \frac{exp\left(\beta_{k0} + \beta_k^T x\right)}{1 + \sum_{l=1}^{k-1} exp(\beta_{l0} + \beta_l^T x)}, k = 1, \ldots, K-1$$

$$Pr(G = K \mid X = x) = \frac{1}{1 + \sum_{l=1}^{k-1} exp(\beta_{l0} + \beta_l^T x)}$$

The estimation is performed by maximum likelihood using the Newton-Raphson algorithm on the expanded parameter vector ϑ (as recommended by ESL for the multiclass case). In each iteration, the gradient and the Hessian are calculated.

If the log-likelihood does not increase, step-halving is applied until the loss decreases, that guarante convergence.

```
class LogisticRegression:
    """
    Attempt of Multiclass Logistic Regression implementation as ESL (Chapter 4.4).
    Using K-1 parameter vectors with the Kth class as reference.
    To handle issues when the Hessian matrix is singular, we use the pseudo-inverse.
    """

    def __init__(self, max_iter=100, tol=1e-6):
        self.max_iter = max_iter # Maximum number of iterations
        self.tol = tol # convergence tolerance on parameter change
        self.theta = None # parameter set of vectors k-1
        self.classes = None # class labels
        self.n_features = None # number of features
        self.K = None # number of classes

    def to_list(self, theta):
        """
        Convert theta into a list of K-1 coefficient vectors.
        """
        betas = []
        start = 0
        for _ in range(self.K - 1):
            betas.append(theta[start:start + self.n_features])
            start += self.n_features

        return betas

    def probability(self, X, theta):
        """
        For classes 1,...,K-1:
            p_k(x) = exp(eta_k) / (1 + sum_{l=1}^{K-1} exp(eta_l))
        For the class K:
            p_K(x) = 1 / (1 + sum_{l=1}^{K-1} exp(eta_l))
        """
        betas = self.to_list(theta) # list of K-1 vectors
        # eta_k = X @ beta_k  for k = 1,...,K-1
        eta = jnp.array([jnp.dot(X, beta) for beta in betas]).T  # (n_samples, K-1)
        exp_eta = jnp.exp(eta) # exp(eta_k)

        denom = 1 + jnp.sum(exp_eta, axis=1, keepdims=True)   # denominator

        probs_no_k = exp_eta / denom # probabilities for classes 1..K-1
        prob_k = 1 / denom # probability for class K

        return jnp.concatenate([probs_no_k, prob_k], axis=1)

    def loss(self, theta, X, y):
        """
        l() = -i log p_{g_i}(x_i;)
        """
        probs = self.probability(X, theta)
        n = X.shape[0]
        real = probs[jnp.arange(n), y] # For each x_i, pick the probability corresponding to
            the class y[i]

        return -jnp.mean(jnp.log(real))

    def fit(self, X, y):
        self.classes = jnp.unique(y)
```

```
        self.K = len(self.classes)
        self.n_features = X.shape[1]

        # Initialise theta = 0
        theta = jnp.zeros((self.K - 1) * self.n_features)
        # Optimize operations with jax
        loss_op = jax.jit(self.loss)
        grad_op = jax.jit(jax.grad(self.loss))
        hessian_op = jax.jit(jax.hessian(self.loss))
        for i in range(self.max_iter):
            loss_old = loss_op(theta, X, y)
            grad = grad_op(theta, X, y)
            Hessian = hessian_op(theta, X, y)

            # Solve H * delta = g
            # assuming Hessian is invertible
            delta = jnp.linalg.solve(Hessian, grad) # Newton update
            theta_new = theta - delta # (quasi Newton methond)

            # Stephalving: if the loss doesnot decrease, we reduce the step by half
            step_factor = 1.0
            while step_factor > 1e-10:
                theta_candidate = theta - step_factor * delta
                loss_new = loss_op(theta_candidate, X, y)
                if loss_new < loss_old:
                    theta_new = theta_candidate
                    break
                step_factor *= 0.5

            # Check we reach tolerance convergence
            if jnp.linalg.norm(theta_new - theta) < self.tol:
                theta = theta_new
                break
            theta = theta_new

            if i % 10 == 0: # step of iterations report
                print(f"Iteration {i}, loss: {loss_old:.6f}")
        self.theta = theta

        return self

    def predict_proba(self, X):
        """
        Return class probabilities for samples in X
        """
        return jnp.array(self.probability(X, self.theta))

    def predict(self, X):
        """
        Predict class labels for samples in X
        """
        probs = self.predict_proba(X)
        return jnp.argmax(probs, axis=1)
```

Metrics:

```
def precision_recall_f1(y_real, y_pred):
    classes = jnp.unique(y_real)
    precisions = []
    recalls = []
    f1s = []

    for c in classes:
        tp = jnp.sum((y_pred == c) & (y_real == c))
        fp = jnp.sum((y_pred == c) & (y_real != c))
        fn = jnp.sum((y_pred != c) & (y_real == c))

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
```

```
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0

        precisions.append(precision)
        recalls.append(recall)
        f1s.append(f1)

    return precisions, recalls, f1s
```

RESULTS

```
metrics_lda = precision_recall_f1(y_test, y_pred_lda)
lda_precision = jnp.mean(jnp.array(metrics_lda[0]))
lda_recall = jnp.mean(jnp.array(metrics_lda[1]))
lda_f1 = jnp.mean(jnp.array(metrics_lda[2]))

print('precision: ', lda_precision, ', recall: ', lda_recall, ', f1: ', lda_f1)
```

```
precision:  0.6754095 , recall:  0.6771378 , f1:  0.67421395
```

Figure 1. Precision, Accuracy and F1-score reported for linear classifier

```
metrics_log = precision_recall_f1(y_test, y_pred_log)
log_precision = jnp.mean(jnp.array(metrics_log[0]))
log_recall = jnp.mean(jnp.array(metrics_log[1]))
log_f1 = jnp.mean(jnp.array(metrics_log[2]))

print('precision: ', log_precision, ', recall: ', log_recall, ', f1: ', log_f1)
```

```
precision:  0.73428255 , recall:  0.73971933 , f1:  0.7354728
```

Figure 2. Precision, Accuracy and F1-score reported for logistic classifier

Given the results of both models, a slight improvement in classification can be observed from the logistic classifier compared to the linear LDA classifier. However, this improvement is not significant, as both models do not appear to have a large difference in precision and recall, giving the impression that some classes are not very well classified. There is also a similar improvement in false positives and false negatives. Furthermore, as part of a second report, feature selection is planned for a dataset with more features, allowing for a better selection of those that best benefit our models.